MINI PROJECT REPORT

Project Phase – 1 REPORT

Submitted by

Parth Solanki [202407020494]

Swayam Tanna [202407020495]

Jay Thakkar [202407020496]

BACHELOR OF TECHNOLOGY

COMPUTER SCIENCE ENGINEERING

Project Mentor

Ms. Pragya Vats

UNITEDWORLD INSTITUTE OF TECHNOLOGY
KARNAVATI UNIVERSITY
Uvarsad, Gandhinagar, Gujarat – 382422

Table of Contents

1. Abstract

2. Introduction

a) Problem Statement
b) Aim and Objectives
c) Scope of the Project

3. System Requirements and Design

a) Hardware Requirements
b) Software Requirements
c) System Architecture
d) Diagrams

4. Implementation

a) Technologies Used
b) Screenshots and Outputs

5. Conclusion

6. Future Scope

7. References

1. Abstract

Rapid urbanisation and the steady growth in the number of vehicles have made road traffic management one of the most pressing challenges for modern cities. Traditional monitoring depends on human operators watching many CCTV feeds at once, which is tiring, error-prone and slow to react to incidents such as accidents, breakdowns, wrong-way driving or sudden congestion. Fixed-timer signals and manual counting also fail to adapt to changing traffic conditions.

This project proposes an **AI-Based Traffic Monitoring and Anomaly Detection System** that analyses video from roadside cameras in real time. The system uses a deep-learning object detector (YOLO) to detect and classify vehicles, a multi-object tracking algorithm to follow every vehicle across frames, and a hybrid anomaly-detection module that combines rule-based checks (stopped vehicle, wrong-way movement, over-speeding, sudden congestion) with an unsupervised machine-learning model that learns normal motion patterns and flags deviations. The results are shown on a dashboard that displays vehicle counts, traffic density and alerts, and every event is saved with a snapshot and timestamp for later review.

This report covers **Project Phase – 1**, which includes problem analysis, requirement study, system design, selection of technologies and the initial implementation plan. The proposed system is expected to reduce the workload of human operators, shorten incident response time and provide data that can support smarter traffic planning.

Keywords: Traffic Monitoring, Anomaly Detection, Computer Vision, Deep Learning, YOLO, Object Tracking, Intelligent Transportation System.

2. Introduction

An Intelligent Transportation System (ITS) uses sensors, communication and data analysis to make road networks safer and more efficient. Among the available sensors, cameras are the most widely deployed because they are inexpensive, already installed at most junctions and capture rich visual information. With recent progress in Computer Vision and Deep Learning, it is now possible for a computer to understand this video automatically: to find every vehicle, follow it, estimate its speed and recognise when something unusual happens.

An **anomaly** in traffic is any event that deviates from the normal flow, for example a vehicle stopped in a live lane, a vehicle travelling against the direction of traffic, an abrupt drop in average speed, a pedestrian on a highway or a collision. Detecting such events quickly is important because the first few minutes after an incident decide whether it ends as a minor delay or becomes a serious accident or a long traffic jam.

a) Problem Statement

Road traffic in Indian cities is dense, mixed (cars, two-wheelers, auto-rickshaws, buses, trucks, cycles and pedestrians share the road) and often undisciplined. Existing monitoring practices have the following limitations:

- **Manual monitoring:** one operator cannot watch dozens of camera feeds continuously, so incidents are noticed late or missed.
- **Fixed-time signal control:** signals follow a pre-set cycle and do not react to the actual queue length or density.
- **Manual or sensor-based counting:** inductive loops and manual surveys are costly, hard to maintain and give only limited data.
- **Delayed incident response:** accidents, breakdowns and wrong-way driving are usually reported by the public, not detected automatically.
- **Unused data:** most recorded footage is only watched after an incident, and no analytics are generated from it.

Therefore, there is a need for an automated, camera-based system that can monitor traffic continuously, measure traffic parameters and detect abnormal events in real time with minimal human effort.

b) Aim and Objectives

Aim: To design and develop an AI-based system that monitors road traffic from video and automatically detects anomalous events in real time.

Objectives:

1. To study existing traffic monitoring and video anomaly detection techniques and identify their limitations.
2. To detect and classify vehicles (car, bike, bus, truck, auto-rickshaw) in video frames using a deep-learning object detector.
3. To track each detected vehicle across frames and estimate its speed and direction of movement.
4. To count vehicles and estimate traffic density and congestion level for each lane or zone.
5. To detect anomalies such as stopped vehicles, wrong-way driving, over-speeding and sudden congestion.
6. To generate real-time alerts and store event logs with snapshots for later analysis.
7. To present all information on an easy-to-use dashboard for traffic operators.

c) Scope of the Project

In scope:

- Processing of live camera streams (RTSP / webcam) and pre-recorded video files.
- Detection, classification, tracking and counting of road vehicles.
- Rule-based and ML-based anomaly detection for the events listed in the objectives.
- A dashboard with live view, statistics, alerts and event history.
- Evaluation on publicly available traffic datasets and sample videos of local roads.

Out of scope (for this phase):

- Automatic number-plate recognition and e-challan generation.
- Direct control of traffic signals or integration with city control-room hardware.
- Large-scale multi-city deployment and edge-hardware optimisation.

Limitations: Accuracy depends on camera angle, lighting, weather and occlusion; the system needs a stable camera position and camera calibration for reliable speed estimation.

3. System Requirements and Design

a) Hardware Requirements

Table 1: Hardware Requirements

Component	Minimum Requirement	Recommended
Processor	Intel Core i5 (8th Gen) / Ryzen 5	Intel Core i7 / Ryzen 7 or higher
RAM	8 GB	16 GB or more
GPU	Not mandatory (runs slowly on CPU)	NVIDIA GPU with 4 GB+ VRAM (e.g. GTX 1650 / RTX 3050) with CUDA
Storage	256 GB SSD	512 GB SSD (for datasets, models and recorded events)
Camera	720p webcam / IP camera	1080p IP camera with RTSP support, fixed mounting
Network	Broadband connection	Wired LAN for IP cameras

b) Software Requirements

Table 2: Software Requirements

Category	Software / Tool
Operating System	Windows 10/11 or Ubuntu 22.04 LTS
Programming Language	Python 3.10 or above
Deep-learning Framework	PyTorch (with Ultralytics YOLO)
Computer Vision Library	OpenCV
Numerical / ML Libraries	NumPy, Pandas, Scikit-learn
Tracking	ByteTrack / DeepSORT
Dashboard / UI	Streamlit or Flask with HTML, CSS and JavaScript
Database	SQLite (can be upgraded to MySQL / PostgreSQL)
Development Tools	VS Code / Jupyter Notebook, Google Colab (for training), Git and GitHub

Functional and Non-Functional Requirements

Functional requirements: (1) accept video input from a camera or file; (2) detect and classify vehicles; (3) track vehicles and assign unique IDs; (4) count vehicles crossing a virtual line; (5) estimate speed and traffic density; (6) detect anomalies and raise alerts; (7) store events with timestamp and snapshot; (8) display results on a dashboard.

Non-functional requirements: (1) near real-time processing (target of at least 15 frames per second on a GPU system); (2) reasonable accuracy under different lighting conditions; (3) modular and easy-to-extend code; (4) simple, clear user interface; (5) privacy-aware design, where only event snapshots are stored and not continuous footage by default.

c) System Architecture

The proposed system follows a layered architecture with four layers, as shown in Fig. 1. Each layer has a clearly defined job and passes its output to the next layer, which keeps the design modular and easy to test.

- **Data Acquisition Layer:** collects video from CCTV / IP cameras, recorded files and public datasets used for training and testing.
- **Pre-processing Layer:** extracts frames, resizes and normalises them and applies basic correction for noise and lighting.
- **AI Analytics Engine:** the core of the system. It contains the vehicle detector, the multi-object tracker and the anomaly-detection module.
- **Output and Presentation Layer:** shows live video with overlays, statistics and alerts on the dashboard and stores events in the database.

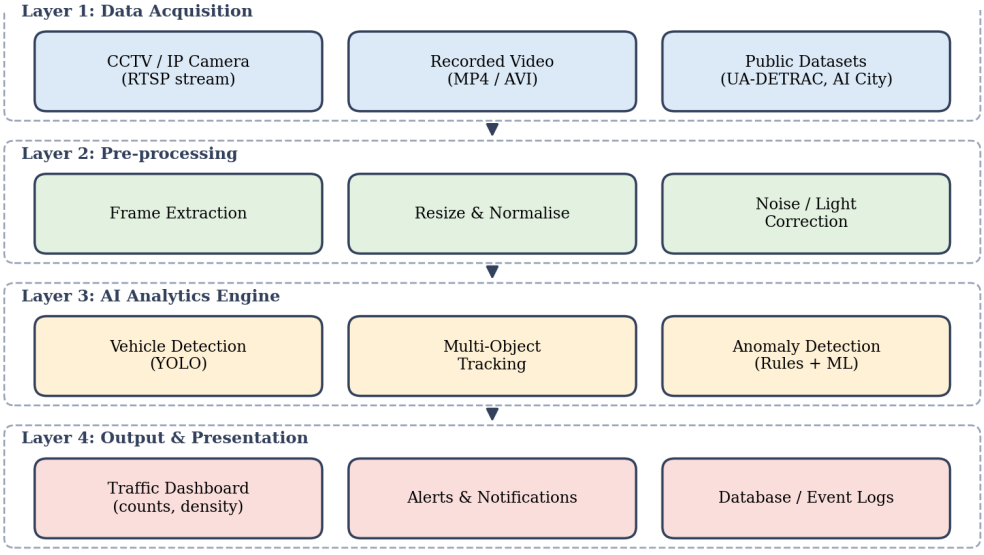


Fig. 1: Layered System Architecture of the AI-Based Traffic Monitoring and Anomaly Detection System

Working of the AI Analytics Engine:

- **Vehicle detection:** each frame is passed to a YOLO model which outputs bounding boxes, class labels and confidence scores. YOLO is chosen because it is a single-stage detector that gives a good balance between speed and accuracy.

- **Tracking:** detections are associated across frames using a tracker such as ByteTrack or DeepSORT, which gives each vehicle a persistent ID. From the movement of the ID's centre point, the system calculates direction, pixel speed and dwell time.
- **Speed and density estimation:** pixel displacement is converted to real-world speed using a perspective (homography) calibration of the road plane. Density is calculated as the number of vehicles inside a region of interest.
- **Anomaly detection:** a hybrid approach is used, described below.

Hybrid anomaly detection. Rule-based checks are simple, explainable and need no training data, but they only catch events that are known in advance. Machine-learning models can catch unusual behaviour that was not explicitly programmed. Combining both gives a more robust system (Fig. 4).

Table 3: Anomaly Types and Detection Logic

Anomaly	Detection logic
Stopped / parked vehicle	Vehicle centre moves less than a small threshold for more than T seconds inside a driving lane
Wrong-way driving	Direction vector of a track is opposite (angle > 135°) to the lane's dominant direction
Over-speeding	Estimated real-world speed is greater than the configured limit
Sudden congestion	Vehicle count in the zone rises above a threshold while average speed falls below a threshold
Unusual motion (ML)	Autoencoder / Isolation Forest trained on normal motion features gives a high anomaly score
Pedestrian on carriageway	A person class is detected inside a vehicle-only region of interest

d) Diagrams

The following diagrams explain the working and the users of the system.

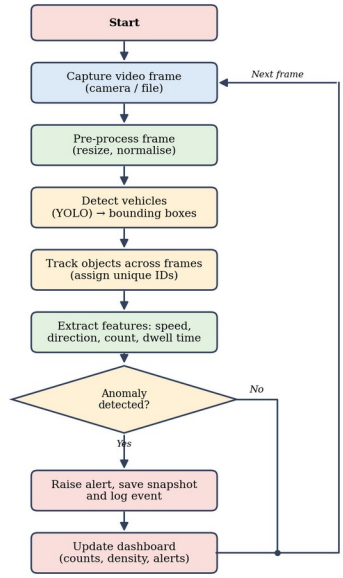


Fig. 2: Flowchart of the Processing Pipeline

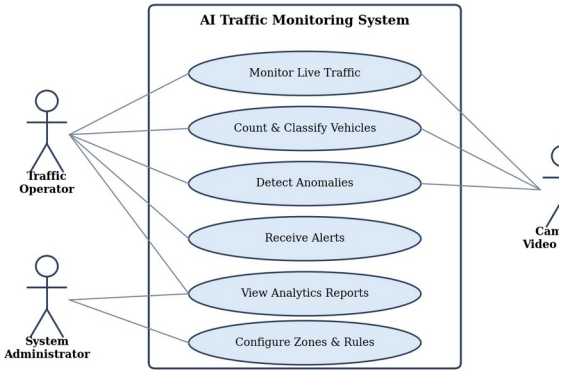


Fig. 3: Use Case Diagram

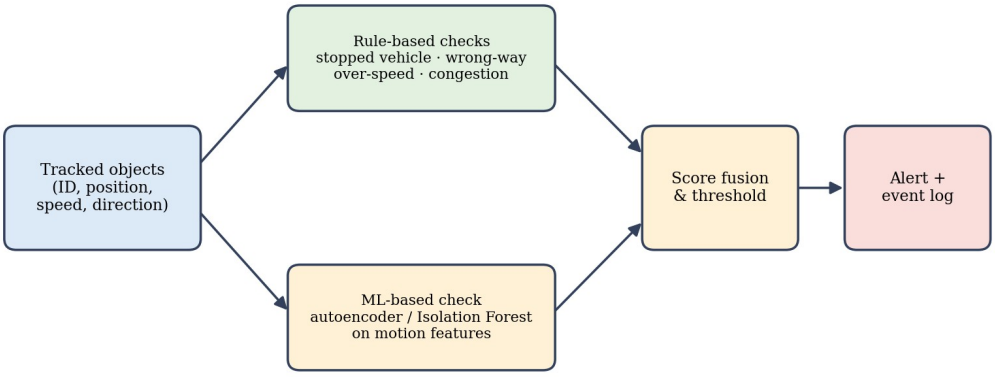


Fig. 4: Hybrid Anomaly Detection Module

4. Implementation

a) Technologies Used

Table 4: Technologies Used

Technology	Purpose in the Project
Python	Main programming language for the complete pipeline
OpenCV	Video capture, frame processing, drawing boxes, zones and overlays
PyTorch + Ultralytics YOLO	Vehicle detection and classification (pre-trained on COCO, fine-tuned on traffic data)
ByteTrack / DeepSORT	Multi-object tracking and ID assignment
NumPy / Pandas	Numerical computation, feature extraction and event tables
Scikit-learn	Isolation Forest and other ML-based anomaly models
Streamlit / Flask	Web dashboard for live view, statistics and alerts
SQLite	Storing event logs, counts and alert history
Google Colab	GPU-based model training and experiments
Git / GitHub	Version control and collaboration

Datasets. Public datasets such as **UA-DETRAC** (vehicle detection and tracking), **COCO** (pre-trained weights) and the **AI City Challenge** traffic anomaly videos are used for training and evaluation. Sample videos from local roads can be added to test performance on Indian traffic conditions.

Implementation steps.

1. Collect and prepare datasets; annotate or convert labels to YOLO format where required.
2. Load a pre-trained YOLO model and fine-tune it on traffic data for the required vehicle classes.
3. Integrate the tracker and verify that IDs stay stable under occlusion.
4. Define regions of interest, counting lines and lane directions through a configuration file.
5. Implement the rule-based anomaly checks and the ML-based anomaly model.
6. Build the dashboard and connect it with the database and the alert module.
7. Test on videos with different lighting, camera angles and traffic density and tune the thresholds.

Core logic (simplified). The following code shows the main idea of detection, tracking and stopped-vehicle detection.

```
from ultralytics import YOLO
import cv2, math

model = YOLO("yolov8n.pt")            # pre-trained detector
cap = cv2.VideoCapture("traffic.mp4")  # or an RTSP URL
history = {}                           # track_id -> list of (x, y, frame_no)
VEHICLES = {2, 3, 5, 7}                # car, motorcycle, bus, truck (COCO ids)
STOP_FRAMES, MOVE_TH = 90, 5           # ~3 s at 30 fps, pixels

frame_no = 0
while cap.isOpened():
    ok, frame = cap.read()
    if not ok:
        break
    frame_no += 1
    res = model.track(frame, persist=True, verbose=False)[0]
    if res.boxes.id is None:
        continue
    for box, tid, cls in zip(res.boxes.xyxy, res.boxes.id, res.boxes.cls):
        if int(cls) not in VEHICLES:
            continue
        x1, y1, x2, y2 = map(float, box)
        cx, cy = (x1 + x2) / 2, (y1 + y2) / 2
        history.setdefault(int(tid), []).append((cx, cy, frame_no))

        # stopped-vehicle rule
        pts = history[int(tid)][-STOP_FRAMES:]
        if len(pts) == STOP_FRAMES:
            d = math.dist(pts[0][:2], pts[-1][:2])
            if d < MOVE_TH:
                print(f"ALERT: vehicle {int(tid)} stopped")  # save snapshot + log
```

b) Screenshots and Outputs

The screenshots below show the expected outputs of the system: the live detection view, the monitoring dashboard and the anomaly alert view.

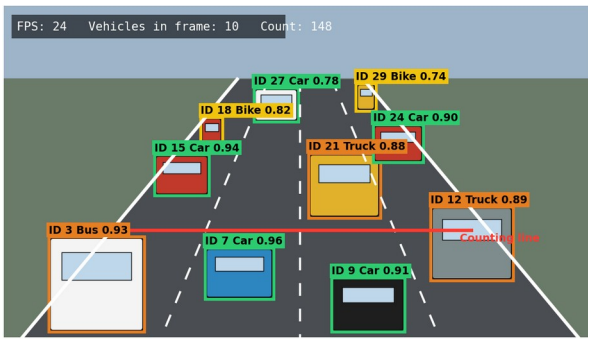


Fig. 5: Vehicle detection with bounding boxes and tracking IDs



Fig. 6: Dashboard with vehicle count, density and congestion level

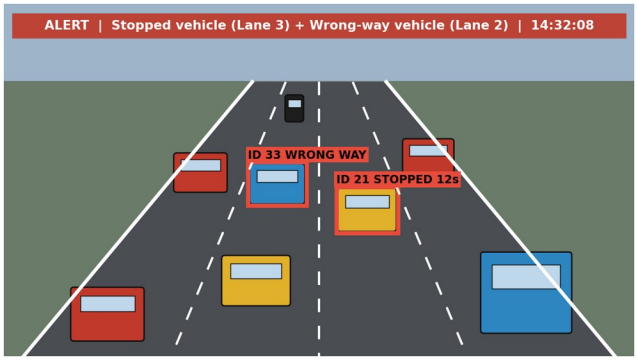


Fig. 7: Anomaly alert with snapshot (stopped vehicle / wrong-way driving)

Performance evaluation. The system will be evaluated using standard metrics: **mAP** and precision/recall for detection, **MOTA / IDF1** for tracking, and **precision, recall, F1-score and false-alarm rate** for anomaly detection, along with processing speed in **frames per second (FPS)**. Final values will be reported after testing is completed in the next phase.

5. Conclusion

In this phase of the project, the problem of manual and delayed traffic monitoring was studied and a complete design for an AI-based traffic monitoring and anomaly detection system was prepared. The proposed layered architecture combines YOLO-based vehicle detection, multi-object tracking and a hybrid rule-based and machine-learning anomaly detection module, supported by a dashboard and an event database.

The design is modular, uses open-source tools and can run on a normal GPU-enabled computer, which makes it affordable and practical for academic and small-scale real-world use. Such a system can reduce the dependency on human operators, help in detecting incidents earlier and give useful traffic data for planning. The remaining work, which includes full implementation, training, testing and tuning on real traffic videos, will be carried out in the next phase of the project.

6. Future Scope

- **Automatic number-plate recognition (ANPR)** for identifying vehicles involved in violations.
- **Helmet, seat-belt and red-light violation detection** for automatic e-challan systems.
- **Accident detection and severity estimation** using video transformers and temporal models.
- **Adaptive traffic-signal control** using reinforcement learning based on live density.
- **Edge deployment** on devices such as NVIDIA Jetson or Raspberry Pi with model optimisation (TensorRT, quantisation).
- **Multi-camera tracking** to follow vehicles across junctions and build city-wide analytics.
- **Traffic prediction** using LSTM / graph neural networks for forecasting congestion.
- **Mobile app and cloud integration** for sending alerts to police, ambulance and the public.
- **Robustness improvements** for night, fog, rain and heavy occlusion using image enhancement and domain adaptation.

7. References

[1] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," IEEE CVPR, 2016.

[2] G. Jocher et al., "Ultralytics YOLO," official documentation, https://docs.ultralytics.com.

[3] Y. Zhang et al., "ByteTrack: Multi-Object Tracking by Associating Every Detection Box," ECCV, 2022.

[4] N. Wojke, A. Bewley and D. Paulus, "Simple Online and Realtime Tracking with a Deep Association Metric (DeepSORT)," IEEE ICIP, 2017.

[5] L. Wen et al., "UA-DETRAC: A New Benchmark and Protocol for Multi-Object Detection and Tracking," Computer Vision and Image Understanding, 2020.

[6] M. Naphade et al., "The AI City Challenge," IEEE CVPR Workshops (Traffic Anomaly Detection track).

[7] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.

[8] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, 2011.